Vaultium

Mário Ferreira

A post-quantum file encryption tool

Contents

AI Usage . . . 4
Cryptographic Algorithms . . . 5
- Symetric . . . 5
- Key Exchange Mechanism . . . 6
- Signatures . . . 7
  - Dilithium . . . 7
  - Falcon . . . 7
  - Sphics . . . 7
- Passwords . . . 9
  - KDF . . . 9
  - HKDF . . . 10

Design . . . 11
- Data encryption . . . 12
  - KEM . . . 12
    - Encapsulate . . . 12
    - Decapsulate . . . 13
  - Key Encryption Key . . . 13
    - Encryption . . . 14
  - Master Key Encryption Key . . . 15
    - Encryption . . . 15
  - File key . . . 16
  - File encryption . . . 16
- Server communication . . . 17
  - Disk encryption . . . 17
  - Disk decryption . . . 18
  - Password change . . . 20

Implementation . . . 21
- Programming Language . . . 21
- Crates . . . 21
  - Authenticated symetric encryption . . . 21
  - KEM and SIGN . . . 21
  - KDF . . . 21
  - HKDF . . . 21
  - Serialization . . . 21
  - CLI arguments . . . 21
- Project structure . . . 21
- Secrets management . . . 22

Bonuses . . . 23
- Clean code (Rust's type system) . . . 23
- PQ TLS . . . 24
  - Full-PQ . . . 25
    - PKI . . . 25
    - Demo . . . 27

Hybrid 28
PKI 28
Demo 29
Self-encryption 30

AI Usage

Throughout this project, AI helped me resolve some challenging compilation errors related to the serialization library I picked (serde), as well as write README files, improve the report's vocabulary, and research PQ-PKI configuration (which arguments to pass to openssl).

No AI models were used to create the cryptographic model, in accordance with the professor's requirements

Cryptographic Algorithms

In order to realise this project, I had to implement different types of cryptographic algorithms (symetric, asymetric and password derivation functions).

The chosen security level for all the post-quantum algorithms is: **V: at least as hard as AES256**

As you know, due to Grover's Algorithm, bruteforce complexity using quantum computes is only $O(\sqrt{n})$, n being the key length. In order to protect ourselves agaisnt quantum computer we simply need to double the size of the keys. As there is no such algorithm that uses a key whose length is **512 bits** (256×2), I chose a key length of **256 bits** to all the symetric algorithms.

Symetric

The best practices recommend to use an authenticated encryption algorithm. I had the choice between **AES-GCM** or **ChaCha20-Poly1305**. I ended up chosing **AES-GCM** as the most significant difference between those two is the fact that **AES-GCM** has hardware support and thus it is faster on most CPUs.

> ❗ **Memorize**
>
> Technically I have chosen **AES-GCM-SIV** which is a variant of **AES-GCM** that is resistant against IV/nonce reuse.
>
> Even though I could have chose the "original" version as, I will explain later, I have identified which IVs/nonces could be hardcoded without any problem. However, **I preferred to take the safest path as it will prevent any kind of implementation error** (reusing IVs where they should not have need hardcoded).

As for the params, I have chosen:

- Key: 256 bits
- Nonce: 96 bits
- Counter: 36 bits
- Tag: 128 bits

Since I have chosen a variant of **AES-GCM**, both nonce and counter sizes have to be 96 bits and 32 bits respectively°. The same logic applies to the tag's size.

> **RFC 8452: AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption**
>
> The AEADs defined in this document calculate fresh AES keys for each nonce. This allows a larger number of plaintexts to be encrypted under a given key. Without this step, AES-GCM-SIV encryption would be limited by the birthday bound like other standard modes (e.g., AES-GCM, AES-CCM RFC3610°, and AES-SIV RFC5297°.
>
> This means that when 2^{64} blocks have been encrypted overall, a distinguishing adversary who is trying to break the confidentiality of the scheme has an advantage of $\frac{1}{2}$. Thus, in order to limit the adversary's advantage to 2^{-32}, at most 2^{48} blocks can be encrypted overall. In contrast, by deriving fresh keys from each nonce, it is possible to encrypt a far larger number of messages and blocks with AES-GCM-SIV.

These constraints leave us with the following:

- Max file size: $2^{36} = 64\ \text{GB}$

In most cases, there are very few files bigger than 64 GB. So, for this reason, this should not be a problem. But, in a real scenario, we should first analyze the kind of data we want to encrypt (based on the victim's profile) and then chose the params that fit the best.

If I had to encrypt files bigger than 64 GB, I would have chosen **ChaCha20-Poly1305** which has a variant that uses a **64-bit counter** (which allows to encrypt $2^{64} = 16\ \text{EB}$) of data.

- aes-gcm-siv (nonce, key, ct,)

Key Exchange Mechanism

As opposed to the symetric cryptographic algorithms, there is not much choice. The chosen algorithm is **ML-KEM** which was the only one described by the teacher.

Bellow, are the sizes of all keys as well as the ciphertext required by **ML-KEM 1024**

Algorithm	SK (B)	PK (B)	PT (B)	CT (B)
ML-KEM 1024	3168 / 32	1568	32	1568

You can also find the performances of the algorithm regarding keypair generation, encryption and decryption[1]

KEYGEN(μs)	ENC(μs)	DEC(μs)
32	42	32

[1]Times are measured on a Intel Haswell 2.3GHz processor

Signatures

For the signatures, there are plenty of choices. Bellow is a detailled comparison for the 3 main signature pq cryptography algorithms .

Dilithium

Algorithm	SK (B)	PK (B)	SIG (B)	Security Level
Dilithium 2	2528	1312	2420	II
Dilithium 3	4000	1952	3293	III
Dilithium 5	4864	2592	4595	V

Algorithm	KEYGEN(μs)	SIGN(μs)	VERIFY(μs)
Dilithium 2	54	144	51
Dilithium 3	111	230	78
Dilithium 5	129	279	122

Falcon

Algorithm	SK (B)	PK (B)	SIG (B)	Security Level
Falcon-512	1281	897	716	I
Falcon-1024	2305	1793	1335	V

Algorithm	KEYGEN(μs)	SIGN(μs)	VERIFY(μs)
Falcon-512	8′640	168	36
Falcon-1024	27′400	343	73

Sphics

Algorithm	SK (B)	PK (B)	SIG (B)	Security Level
SPHINCS$^{+}_{-}$128s	64	32	7856	I
SPHINCS$^{+}_{-}$128f	64	32	17088	I
SPHINCS$^{+}_{-}$192s	96	48	16224	III
SPHINCS$^{+}_{-}$192f	96	48	35664	III
SPHINCS$^{+}_{-}$256s	128	64	29792	V
SPHINCS$^{+}_{-}$256f	128	64	49856	V

Algorithm	KEYGEN(μs)	SIGN(μs)	VERIFY(μs)
SPHINCS^{+}128s	18′658	141′392	209
SPHINCS^{+}128f	295	6′965	579
SPHINCS^{+}192s	28′249	270′738	327
SPHINCS^{+}192f	438	11′884	820
SPHINCS^{+}256s	18′452	241′700	451
SPHINCS^{+}256f	1′146	24′287	825

Since I have chosen the **V security level**, I ended up with a few choices:

Algorithm	SK (B)	PK (B)	SIG (B)	Security Level
Dilithium 5	4864	2592	4595	V
Falcon-1024	2305	1793	1335	V
SPHINCS^{+}256s	128	64	29792	V
SPHINCS^{+}256f	128	64	49856	V

Algorithm	KEYGEN(μs)	SIGN(μs)	VERIFY(μs)
Dilithium 5	129	279	122
Falcon-1024	27′400	343	73
SPHINCS^{+}256s	18′452	241′700	451
SPHINCS^{+}256f	1′146	24′287	825

As you can see, **Falcon-1024** (**FN-DSA**), is the algorithm that has the best balance between signing, verifying and smallest signatures. The keygen "huge" performance issues might seem a problem, but keys are only generated once in the whole server's lifetime.

However, it was not officialy released by the NIST since the **FN-DSA for JOSE and COSE** RFC is still a draft[2].

Thus, the second best choice is Dilithium 5 and it was the chosen algorithm.

[2] https://datatracker.ietf.org/doc/draft-ietf-cose-falcon/

Passwords

When the client is launched, it selects a random word from a dictionnary and uses that word to derivate a key that will then be used to protect the **Master Key Encryption Key** (more details in the Master Key Encryption Key section).

The **Master Key Encryption Key** is a random 256-bit key what will be used to encrypt the files on the victim's machine.

Thus, both **KDF** (for low-entropy input) and **HKDF** (for high-entropy input) were used to derive those two keys mentioned above.

KDF

As the goal of a good KDF is to prevent the bruteforce, specially on GPUs, computing a key based on a low-entropy input such as a password has to be slow.

Currently, **Argon2** is the most recommended KDF algorithm and for this reason **it was chosen as the implemented KDF for this project**.

Regarding the parameters of **Argon2**, I had two choices: either use a tool such as kratos° or argon2-cffi° or choosing the params by myself.

Initially, I used those 2 tools but when I used the params given by them in my code I had huge performance differences. It might be due the **Argon2** implementation in Go (kratos), Python (argon2-cffi) and Rust (my code). I ended up finding the **FFC 9106 Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications**[3] that recommends **Argon2** params based on different scenarios.

Bellow, you can find the two recommended params based on that same RFC:

Configuration	Type	Iterations (t)	Lanes (p)	Memory (m)	Salt	Tag
First recommended	Argon2id	1	4	2^{21} (2 GiB)	128-bit	256-bit
Second recommended	Argon2id	3	4	2^{16} (64 MiB)	128-bit	256-bit

I have select the first recommendation and tested in my code, but the hashing of a password was way too fast. So, I incremented the **iterations** parameter until the elapsed time for hashing a password was about **3s**.

[3] https://www.rfc-editor.org/rfc/rfc9106.html#name-parameter-choice

Here is the code snippet I used to find **Argon2**'s parameters:

```rust
use std::time::Instant;

// 128-bit salt
let salt = Salt::generate();
let mut key = SymetricKey::new();
let params =
    Params::new(2_097_152, 2, 4, Some(256 / 8)).expect("Failed to create Argon2 params");

println!(
    "Memory: {} KiB, Iterations: {}, Parallelism: {}",
    params.m_cost(),
    params.t_cost(),
    params.p_cost()
);

let argon2 = Argon2::new(Algorithm::Argon2id, Version::V0x13, params);

let start = Instant::now();
argon2.hash_password_into(word.as_bytes(), salt.as_ref(), key.as_mut())?;
println!("Hashing took: {:?}", start.elapsed());
```

Finally, after a few attempts, the final results were (tested on my machine using cargo's release target):

Memory (KiB)	Iterations (t)	Lanes (p)	Elapsed time (s)
2097152	2	4	2.92558099

HKDF

As explained above, a HKDF algorithm was used in order to derive a key based on the **Master Key Encryption Key** (which is a high-entropy input) for each file on the victim's hard-drive by using the file path as the additional information parameter.

> **Important**
>
> The salt is set to 0^{128} as recommended in the slides (password's chapter at page 11).

Thus, **HMAC-SHA256** is the final choice since it is the most recommended HKDF algorithm.

Design

In this chapter, I will explain the design I have chose. That is, how I have used passwords to derivate a key, how does the client ensure that the keys generated by the server where not modified and how both the client and server ensure that their communications remains confidential.

Regarding the later, I have decided to use a KEM algorithm as described in the Key Exchange Mechanism section. The server's KEM public key is copied into the client's binary (into a static variable copied at comptime). This allows the client to encapsulate the server's public key, obtain a shared secret and derivate a symetric key that will then be used to encrypt data to the server.

Then, thanks to the cipher text generated during KEM encapsulation and sent by the client, the server will be able to derivate the same symetric key and decrypt the data. Also, since **AES-GCM-SIV** is used, the server will also check if the tag is valid.

As for the keys signatures, the client also has the server's singning public key copied into its own binary. That key will be used when the client will ask the server to either decrypt a single file or the whole disk. In this case, the server will first sign the required keys and send both the keys and the respective signatures.

The client-server communication is described in details in this section. There, I have explained how the communication is encrypted in both ways (client and server side).

Data encryption

KEM

A KEM algorithm was used some parts of the communication between the client and server. Bellow, you will find the **encapsulate** and **decapsulate** diagrams used by both the client and server respectively in order to obtain a **256-bit** symetric key.

The info used by the **HKDF** function is an hardcoded string and a **UUID**. This **UUID** is a random identifier given to the client during compilation. It is simply used by the server to store the keys of each client in a different directory.

Since the **UUID** is generated using the **UUID** rust's crate and that input validation is made by the cli parsing arguments crate I have chose (Serde) there is **no separation of domains problems**.

Here is a code snippet of the struct that defines the client's CLI arguments:

```rust
#[derive(Parser, Debug)]
#[command(version, about, long_about = None)]
struct Args {
    // ...
    #[arg(long)]
    id: uuid::Uuid,
}
```

Encapsulate

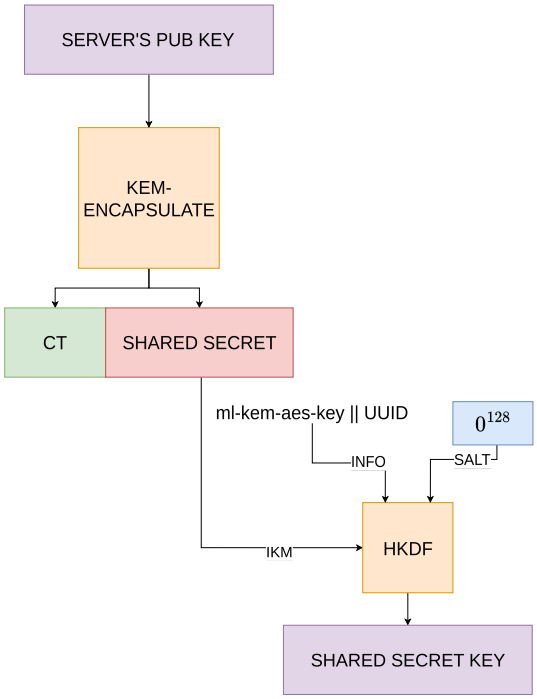

Figure 1: KEM encapsulation

Decapsulate

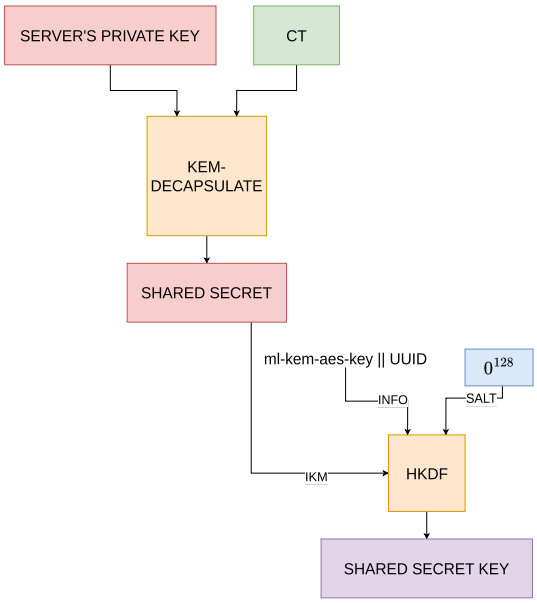

Figure 2: KEM decapsulation

Key Encryption Key

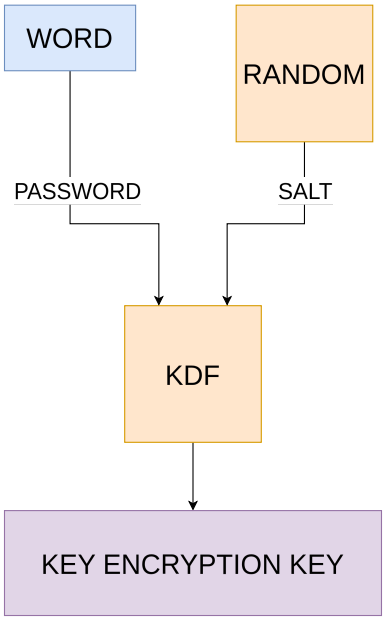

Figure 3: "Key Encryption Key generation"

The **Key Encryption Key** is a key generated from a password that will be used to encrypt the **Master Key Encryption Key** which generates a different key per file. The password is a random one that the client fetches randomly from a dictionnary that was copied into its binary.

The dictionnary contains passwords of 8-15 characters. The salt is a random **128-bit** value generated the same way as the Master Key Encryption Key (CSPRNG seeded by a TRNG).

The KDF parameters are decribed in details in the KDF section.

Encryption

On the client side, once the **Key Encryption Key** is generated, the server's kem public key is then encapsulated and a **256-bit** key is generated from the **shared secret**. That key is used to encrypt the **Key Encryption Key**.

> **Important**
>
> Due to the fact that **KEM** encapsulation is non-deterministic, the **shared secret will always be a different value**. And so, since the **shared secret** is used to generate a **256-bit** key, that key will always be different.
>
> For this reason, I have set the nonce to 0^{128}. I could have generated a random **128-bit** nonce just like for the **Master Key Encryption Key** but I wanted to simplify the design and also study in more details where nonces/ivs could be hardcoded without any problem.

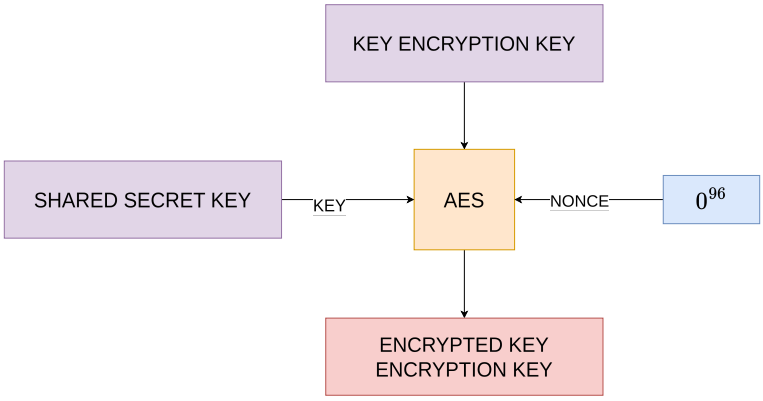

Figure 4: "Key Encryption Key encryption"

Master Key Encryption Key

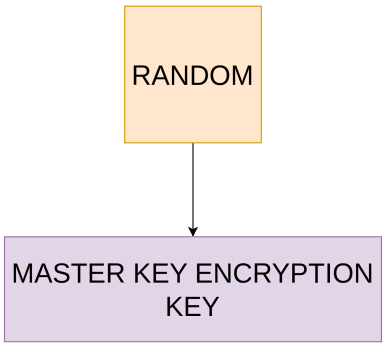

Figure 5: "Master Key Encryption Key generation"

As already explained, the **Master Encryption Key** is a **256-bit** random key generated by Rust's rand::rng() RNG°. It is a **CSPRNG** seeded by a **TRNG** (OsRng)[4].

It is the **most important key of the system**. It is used to generate unique keys for each file during both file encryption and decryption. If it is leaked, an attacker can decrypt the whole victim's encrypted disk.

Encryption

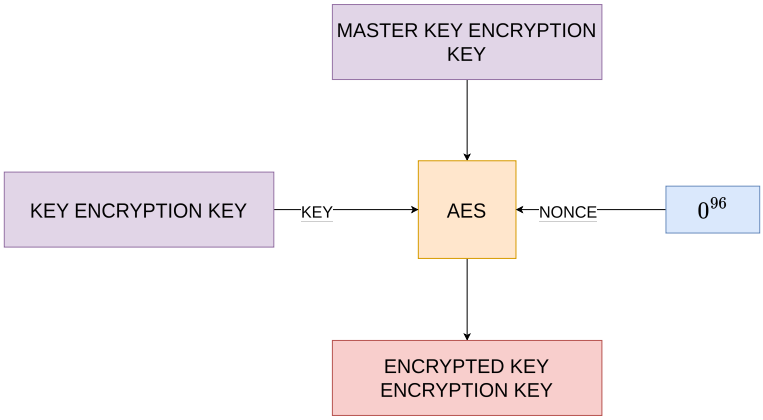

Figure 6: "Master Key Encryption Key encryption"

> **Important**
>
> For the same reason stated in Key Encryption Key encryption section, the nonce is also set to 0^{128}

The **Master Key Encryption Key** is encrypted using the **Key Encryption Key** (generated key from a random password) as the symetric key. The ciphertext and the tag are sent to the server which stores the encrypted **Master Key Encryption Key** in a file.

[4] https://rust-random.github.io/book/guide-rngs.html#state-and-seeding

File key

On each file system, each file has a unique path. Therefore, I used the **relative path** to generate a unique key per file based on the **Master Key Encryption Key**. I had to use the relative path so that both the client and the server generate the same **file key** even if they do not have the exact file system structure (same folders on both disks).

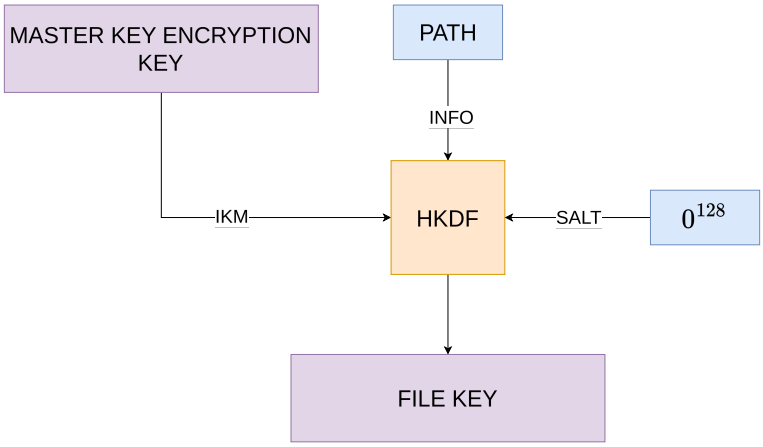

Figure 7: File key generation

File encryption

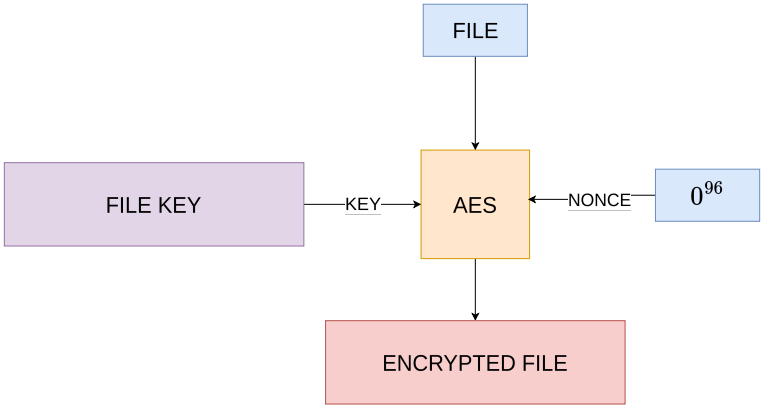

Figure 8: File encryption

Finally, when it comes to the file encryption, I simply use the generated **file key** and encrypt the file using **AES-GCM-SIV**.

> **Important**
>
> Since the **file key** is an unique **256-bit** value, the nonce is also set to 0^{128}.

Server communication

All communication between the client and server is done via HTTP. The main reason for this is that I've always wanted to create a web API in Rust to see if it was really a good solution compared to other programming languages and web frameworks such as **Laravel (PHP)**, **Ruby On Rails (Ruby)**, or **Phoenix (Elixir)**.

Furthermore, I knew that by using a simple web framework and a serialization library such as serde, I wouldn't spend most of my time trying to serialize keys and then send them to a TCP socket. This process in HTTP was actually quite simple, as I had already done it in Rust.

By using HTTP, I could also setup a post-quantum PKI since rustls has post-quantum cryptography support[5].

Disk encryption

Initially, the client generates a **Key Encryption Key** from a random password taken from a dictionnary. Then it encapsulates the server's public key in order to generate a shared secret and a ciphertext. A **256-bit** is then derived from that shared secret using and hardcoded string and the user's HTTP token (a simple **UUID** passed as "Authorization: Bearer <TOKEN>").

The **Key Encryption Key** is encrypted using the **256-bit** key derived from the shared secret. Both encrypted **Key Encryption Key** and the cipher text (ml-kem ciphertext) are sent to the server so it can decapsulate the ciphertext using its own private key and derive the same **256-bit** key to decrypt the **Key Encryption Key**.

The client then proceeds to generate a random **256-bit** key (**Master Key Encryption Key**). This key is encrypted using the **Key Encryption Key** and sent to the server.

Finally, the client starts to generate a different key per file and encrypts the disk.

[5] https://crates.io/crates/rustls-post-quantum

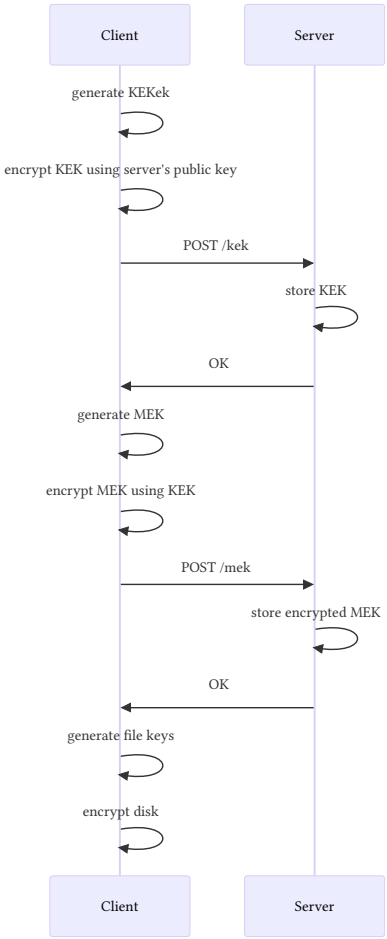

Figure 9: Disk encryption

Disk decryption

Since I wanted to encrypt the communication between the client and the server, I used the server's public key to encrypt (derive a key based on a shared secret) the traffic coming from the client towards the server.

However, there was a problem: the traffic from the server was not encrypted. I could have used the same technique as above by generating a keypair for the client and sending the client's public key to the server but the **problem is that the client's private key should be stored on the victim's disk**. Disk would leak all the information sent from the server.

The only solution I found for this problem as to generate an emepheral **256-bit** key on the client side and send it to the server (encrypted with the server's public key). Then, the server uses that **emepheral key** to encrypt its traffic to the client. This includes the **File Key** and the **Master Key Encryption Key** required to either decrypt a single file or the whole disk by the client.

Hence, the communication between the client and the server is encrypted in both ways (without having to setup a post-quantum PKI and use HTTPS).

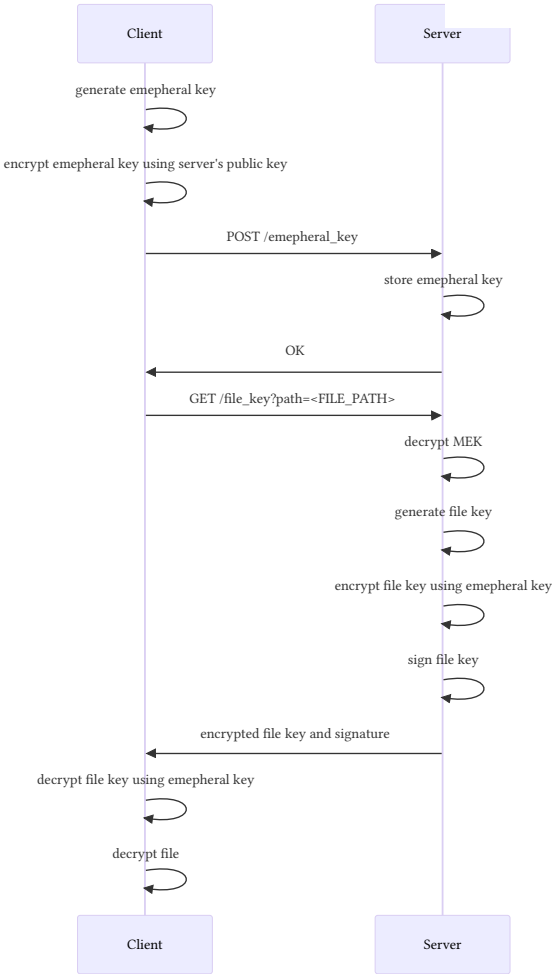

Figure 10: Single-file decryption

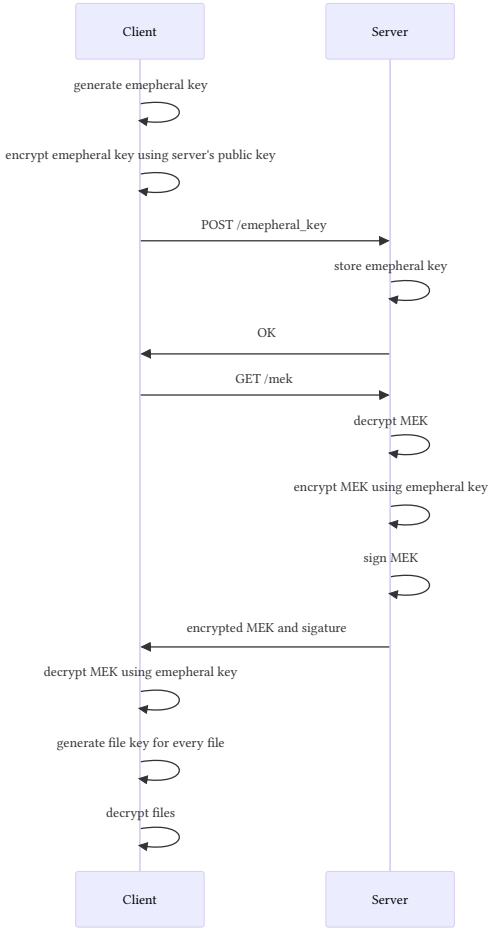

Figure 11: Disk decryption

Password change

When the server asks the client to change the password, the client simply makes the same first steps as when generating the **Key Encryption Key**.

One of the requirements of this project was to, when changing password, not leak any data regarding the encrypted files. Whichs means that the files must not be decrypted and then decrypted.

Also, the password change time complexity must be linear (somewhere around $O(1)$). Initially, I though of having a key rotation system of the **Master Key Encryption Key** where all the keys where stored on the server and the files were encrypted just like a onion for each rotated key.

The problem with that approach was that file decryption was way too slow.

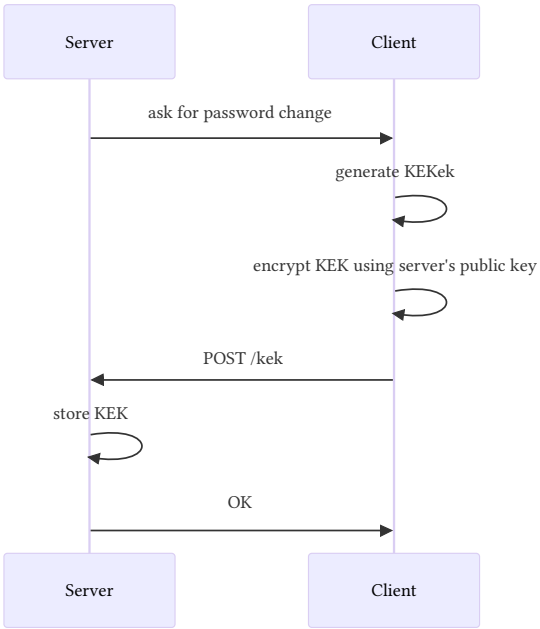

Figure 12: Password change

Implementation

Programming Language

The programming language I selected to implement this project is Rust. It is one of my favorite programming languages.

The main reasons I chose it are as follows:

- A vast ecosystem of crates
- An excellent type system that helps prevent implementation errors
- To improve my Rust skills, as the code for my bachelor's thesis will be written in this language

Crates

In this section, I will briefly explain why I chose certain crates.

Authenticated symetric encryption

AES-GCM-SIV: recommended by the well-known "Awesome Rust Cryptography" website[6]

KEM and SIGN

rustpq/pqcrypto: Rust's bindings to the original C implementation of ML-KEM and Dilithium 5

KDF

argon2: listed in "Awesome Rust Cryptography"[7]

HKDF

HKDF: listed in "Awesome Rust Cryptography"[8]

Serialization

serde: the defacto Rust's serialization crate

CLI arguments

clap: the defacto Rust's CLI arguments parsing crate

Project structure

The project is split into 3 crates:

- client: Binary crate containing the client code
- server: Binary crate containing the server code
- lib: Lib crate containing all cryptographic code. Used by both the client and the server

The most important crate is **lib**, because it contains the cryptographic code. I created it to reduce code duplication and, above all, **to avoid implementation errors in my code**.

This way, I ensure that the client and server use the same keys, parameters, algorithms, etc.

[6] https://cryptography.rs/#symmetric-cryptography
[7] https://cryptography.rs/#password-hashing-functions
[8] https://cryptography.rs/#hash-functions-and-friends

In terms of dependency management, I used a Cargo workspace[9]. The advantage of this is that dependencies can be defined at the project level, and the **same versions of dependencies** can be reused each time the project is created.

Secrets management

When creating any kind of cryptography-related project, one of the most important things is to properly manage secrets in memory.

For most of the sensitive data, I clean the memory when the variables holding that data leave a scope. This is done by the Zeroize[10] create as you can see bellow:

```rust
#[derive(Serialize, Deserialize, Zeroize, ZeroizeOnDrop, Clone)]
pub struct SymetricKey {
    pub data: KeyData,
}
```

Unfortunately, it is not possible to clean up post-quantum cryptographic secrets if you use the library I chose, as you can see here[11] and here[12].

[9] https://doc.rust-lang.org/cargo/reference/workspaces.html
[10] https://github.com/RustCrypto/utils/tree/master/zeroize
[11] https://github.com/rustpq/pqcrypto/issues/31# issuecomment-983515470
[12] https://github.com/rustpq/pqcrypto/pull/91#pullrequestreview-3151942386

Bonuses

Clean code (Rust's type system)

Rust's type system is very powerful. That's why I designed my code to use it in order to avoid silly implementation errors.

First, I created custom types for each type of key. For example, the following **SymetricKey** structure represents a **256-bit** key. I did the same for nonces, salt, password, etc.

```rust
pub const SYMETRIC_KEY_LENGTH: usize = 256 / 8;
pub type KeyData = [u8; SYMETRIC_KEY_LENGTH];

#[derive(Serialize, Deserialize, Zeroize, ZeroizeOnDrop, Clone)]
pub struct SymetricKey {
  pub data: KeyData,
}
```

This ensures that sensitive data is handled in the same way throughout the code.

I also created generic traits that define how certain keys can be encrypted (using a symmetric key or a shared secret key derived from a KEM public key). For each of these, I can also define the corresponding encrypted version of the key. Thanks to this type association, the compiler guided me in detecting any implementation errors.

```rust
pub trait AsymetricEncryptable<T>: Serialize
  where T: AsymetricEncryptedKey + DeserializeOwned {
    fn encrypt(&self, pk: &PublicKey, info: &[u8]) -> anyhow::Result<T> {}
}
pub trait AsymetricDecryptable<T>: Serialize
  where T: DecryptedKey + DeserializeOwned {
    fn decrypt(&self, sk: &SecretKey, info: &[u8]) -> anyhow::Result<T>
      where Self: AsymetricEncryptedKey {}
}
pub trait SymetricEncryptable<T>: Serialize
  where T: SymetricEncryptedKey + DeserializeOwned {
    fn encrypt(&self, key: &SymetricKey) -> anyhow::Result<T> {}
}
pub trait SymetricDecryptable<T>: Serialize
  where T: DecryptedKey + DeserializeOwned {
    fn decrypt(&self, key: &SymetricKey) -> anyhow::Result<T> where Self: SymetricEncryptedKey {}
}
```

For example, here, I tell the compiler that a **Master Key Encryption Key** can be encrypted using a symmetric algorithm (in this case, the **Key Encryption Key**) and that an encrypted **Master Key Encryption Key** can also be decrypted using a symmetric algorithm to obtain a **Master Key Encryption Key**.

```rust
impl SymetricEncryptable<EncryptedMasterEncryptionKey> for MasterEncryptionKey {}
impl SymetricDecryptable<MasterEncryptionKey> for EncryptedMasterEncryptionKey {}
```

PQ TLS

Even though is as not 100% required to setup a PQ PKI and use PQ TLS, I still have done it for the sake of learning what changes from a traditional PKI/TLS.

Initially, I wanted to only setup a simple nginx instance that served a silly HTML page through PQ-TLS. However, I then found out that rustls supported an hybrid post-quantum key exchange algorithm[13].

For this reason, I created two different setups: one full-pq using an nginx proxy (and full-pq certificates) and a curl container compiled to support post-quantum algorithms during TLS handshake and a second one that also uses an nginx proxy (with "standard" certificates) and the client properly setup to perform HTTPS requests using that hybrid algorithm.

Both the nginx and curl containers are deployed using a **compose.yaml** file as explained bellow.

Finally, regarding the hybrid PQ-PKI version, the ROOT CA certificate is copied into the client's binary. It was the easiest way to setup **rustls**.

> **Info**
>
> All the generated certificates are delivered with the report and the source code so that you can simply build and deploy the containers in order to test the PQ-TLS setup.

[13] https://docs.rs/rustls/latest/rustls/manual/_05_defaults/index.html#about-the-post-quantum-secure-key-exchange-x25519mlkem768

Full-PQ

PKI

Before generating the certificates, I had to build and install the openssl pq-algorithms provider.

First, I had to install **liboqs**[14] required by **oqs-provider**[15] (the pq-algorithms openssl provider).

Then, I have installed **oqs-provider** as follows:

```sh
git clone https://github.com/open-quantum-safe/oqs-provider && cd oqs-provider
./scripts/fullbuild.sh
```

Once both **liboqs** and **oqs-provider** were installed, I proceeded to generate the certificates. Notice that, for simplicity purposes, I have not setup a database for the certificates emissions like I did in **CRY PW04**.

The openssl configuration files will not be described here. If you want further details take a look at them under **nginx/certs/config/**

As stated in the beginning of the document, I chosed **falcon1024** as the certificates' signature algorithm.

[14] https://github.com/open-quantum-safe/liboqs
[15] https://github.com/open-quantum-safe/oqs-provider

```sh
root="root-ca"
web_server="nginx"

openssl req -x509 -new -nodes \
  -newkey falcon1024 \
  -provider base \
  -provider default \
  -provider oqsprovider \
  -provider-path ./oqs-provider/_build/lib \
  -keyout data/full-pq/$root.key \
  -out data/full-pq/$root.crt \
  -days 1825 \
  -config config/$root.cnf

openssl genpkey \
  -algorithm falcon1024 \
  -provider base \
  -provider default \
  -provider oqsprovider \
  -provider-path ./oqs-provider/_build/lib \
  -out data/full-pq/$web_server.key

openssl req -new \
  -key data/full-pq/$web_server.key \
  -out data/full-pq/$web_server.csr \
  -config config/$web_server.cnf

openssl x509 -req \
  -in data/full-pq/$web_server.csr \
  -CA data/full-pq/$root.crt \
  -CAkey data/full-pq/$root.key \
  -CAcreateserial \
  -out data/full-pq/$web_server.crt \
  -days 365 \
  -extfile config/$web_server.cnf \
  -extensions server_reqext \
  -provider base \
  -provider default \
  -provider oqsprovider \
  -provider-path ./oqs-provider/_build/lib
```

Unfortunately, there were no step-by-step tutorials/documentation on the web regarding the creation of a PQ-PKI. So I had to spend some time reading the openssl documentation[16] and proceeding by trial and error.

[16] https://docs.openssl.org/3.5/man7/EVP_PKEY-ML-KEM/#common-parameters

Demo

To verify that the configuration works, I created a **compose.yaml** file that launches an nginx container and a curl container.

The Docker images come from **open-quantum-safe/oqs-demos/nginx**[17] and **open-quantum-safe/oqs-demos/curl**[18] dockerfiles.

I modified them slightly to change the algorithms and remove some steps that were unnecessary for this demonstration.

```yaml
services:
  nginx-hybrid:
    container_name: nginx-hybrid
    build:
      context: ./nginx/
      dockerfile: ./Dockerfile
    ports:
      - "4433:4433"
    volumes:
      - ./nginx/certs/data/hybrid/nginx.crt:/opt/nginx/pki/server.crt:ro
      - ./nginx/certs/data/hybrid/nginx.key:/opt/nginx/pki/server.key:ro
    extra_hosts:
      - "host.docker.internal:10.5.0.1"
    networks:
      caa-miniproject-hybrid:
        ipv4_address: 10.5.0.2
networks:
  caa-miniproject-hybrid:
    driver: bridge
    ipam:
      config:
        - subnet: 10.5.0.0/16
          gateway: 10.5.0.1
```

Once you have deployed the containers with the **docker-compose up –build** command, you can connect to the curl container and type the following:

```sh
curl -vvv https://10.5.0.2:4433/full-pq --cacert /opt/oqssa/root-ca.crt
```

This connects to the nginx container through TLS using the pq-certificates created above. The nginx container will simply return a string.

[17] https://github.com/open-quantum-safe/oqs-demos/tree/main/nginx
[18] https://github.com/open-quantum-safe/oqs-demos/tree/main/curl

Hybrid

Despited of the fact that full-pq TLS works, the industry is starting to first use an hybrid version of it[19].

One of the most used hybrid versions of PQ-TLS is to use both **ECDHE** and **MLKEM** for the key agreement in TLSv1.3[20]. This is also the case of the **rustls** crate I used to make HTTPS requests from the client.

PKI

Regarding the PKI setup, there was no special openssl provider to install as the certificates are "standard" (not post-quantum).

```sh
root="root-ca"
web_server="nginx"

# Generate Root CA with ECDSA (required by rustls)
openssl ecparam -genkey -name prime256v1 -out data/hybrid/$root.key

openssl req -x509 -new -nodes \
  -key data/hybrid/$root.key \
  -out data/hybrid/$root.crt \
  -days 1825 \
  -config config/$root.cnf

openssl ecparam -genkey -name prime256v1 -out data/hybrid/$web_server.key

openssl req -new \
  -key data/hybrid/$web_server.key \
  -out data/hybrid/$web_server.csr \
  -config config/$web_server.cnf

openssl x509 -req \
  -in data/hybrid/$web_server.csr \
  -CA data/hybrid/$root.crt \
  -CAkey data/hybrid/$root.key \
  -CAcreateserial \
  -out data/hybrid/$web_server.crt \
  -days 365 \
  -extfile config/$web_server.cnf \
  -extensions server_reqext

# Verify
openssl verify -CAfile data/hybrid/$root.crt data/hybrid/$web_server.crt
```

[19] https://blog.cloudflare.com/pq-2024/#ml-kem-768-and-x25519
[20] https://www.ietf.org/archive/id/draft-ietf-tls-ecdhe-mlkem-03.html

Demo

Just for the full-pq version, simply deploy the containers described in the **compose.yaml** file:

```yaml
services:
  nginx-hybrid:
    container_name: nginx-hybrid
    build:
      context: ./nginx/
      dockerfile: ./Dockerfile
    ports:
      - "4433:4433"
    volumes:
      - ./nginx/certs/data/hybrid/nginx.crt:/opt/nginx/pki/server.crt:ro
      - ./nginx/certs/data/hybrid/nginx.key:/opt/nginx/pki/server.key:ro
    extra_hosts:
      - "host.docker.internal:10.5.0.1"
    networks:
      caa-miniproject-hybrid:
        ipv4_address: 10.5.0.2
networks:
  caa-miniproject-hybrid:
    driver: bridge
    ipam:
      config:
        - subnet: 10.5.0.0/16
          gateway: 10.5.0.1
```

Regarding the client, I had to configure **rustls** so that it could use the pq hybrid algorithm and the ROOT CA certificate. If you want more details, check out the **src/client/main.rs** file.

Then, once nginx container is running, launch the client with the **--base-url https://127.0.0.1:4433** argument.

Self-encryption

Real-world ransomwares protect themselves against reverse engineering techniques (either static or dynamic).

One of the most accurate techniques is self-encryption. Some malwares even partially decrypt the binary and then encrypt. This reduces the time frame where the code is loaded into memory and can be extracted for forensics analysis.

Even though there are open source tools that do it (or at least pack and unpack .elf binaries) such as UPX[21], I decided to implement my own in order to learn more about malware protection techniques.

In my case, encryption is done thanks to the **packer/** crate. It generates a random **256-bit** key, obfuscates it using a XOR and rotate and a swap techniques. I do know that these are really simple obfuscation techniques but Unfortunately I did not have enough time to implement more advanced techniques.

On the other hand, the decryption is done by the **unpacker/** crate. During compilation, I copies the encrypted binary into its own and, when executed, it deobfuscates the key and starts to **decrypt the entire file**.

Then, the decrypted binary loaded into memory is copied to an executable memory using **memfd**.

Here is how to encrypt and decrypt the client's binary:

Encrypt:

```sh
# ensure that the client binary exists
pushd client && cargo build --release && popd
cd packer
cargo run --release
```

Decrypt:

```sh
cd unpacker
cargo run --release -- # SAME CLIENT ARGUMENTS
# EXAMPLE
cargo run --release -- --base-url http://127.0.0.1:8085 --dictionnary ./data/dictionnary.txt --work-dir ./data/files/ --id 2c6fd123-ef03-4246-b020-e61a364168b7 --mode encrypt

# USAGE FUNCTION
cargo run --release -- --help
```

[21] https://upx.github.io/